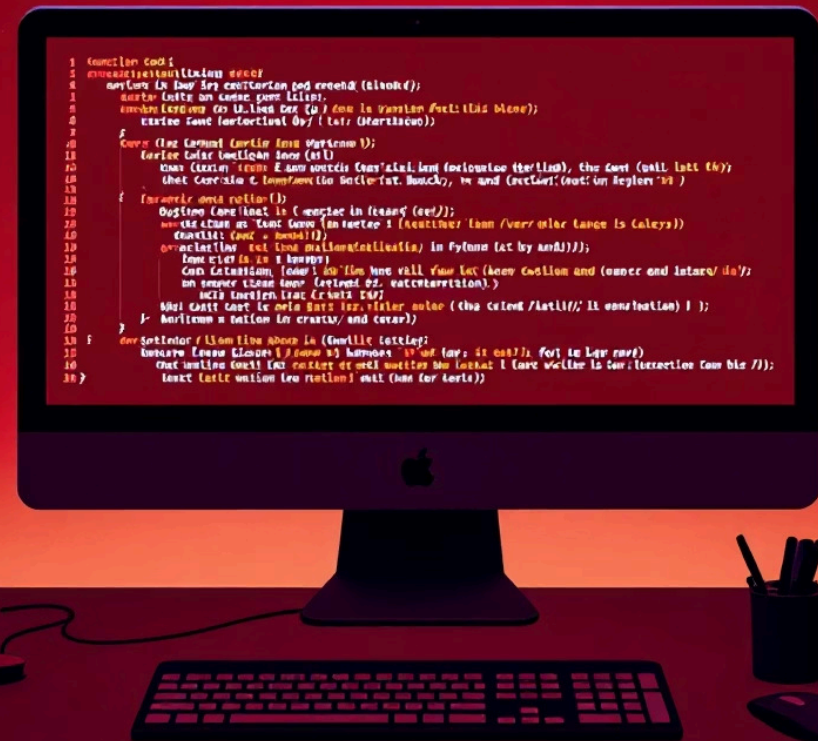


Aula 1 - Entendendo a Estrutura de Programas com Módulos

Uma jornada prática para organizar e modularizar seu código Python de forma profissional



Objetivos da Aula



Conceito de Módulo

Compreender o que são módulos em Python e como eles funcionam



Criação e Organização

Aprender a criar módulos e organizá-los em diferentes arquivos



Navegação de Diretórios

Entender estruturas de pastas e navegar entre elas para acessar módulos



Raciocínio Modular

Desenvolver o pensamento de modularização para código mais eficiente

Introdução – O que é um módulo?

Um **módulo em Python** é qualquer arquivo `.py` que contém código reutilizável — funções, variáveis ou classes.

Eles são usados para **organizar programas grandes** em partes menores, facilitando a manutenção e a leitura do código.

Por que usar módulos?

- Dividir o código em partes menores e compreensíveis
- Reutilizar funções em outros programas
- Facilitar a manutenção do sistema
- Promover colaboração entre desenvolvedores



Seu Primeiro Módulo Python

Exemplo simples de módulo:

```
# arquivo: saudacoes.py

def ola():
    print("Olá, seja bem-vindo!")

def tchau():
    print("Até logo!")
```

📄 **Pronto!** Esse arquivo já é um módulo Python. Qualquer arquivo `.py` com código pode ser importado como módulo.



Sistema Escolar

`alunos.py` — funções relacionadas a alunos



Gestão de Notas

`notas.py` — funções relacionadas a notas



Arquivo Principal

`main.py` — arquivo que organiza tudo



Criando e Acessando Módulos

Passo 1: Criar o módulo

Crie um arquivo `funcoes.py` com funções simples:

```
def saudacao(nome):  
    print(f"Olá, {nome}!")  
    print("Seja bem-vindo(a).")  
  
def somar(a, b):  
    return a + b
```

Passo 2: Importar e usar

Crie outro arquivo chamado `main.py`:

```
# Forma 1: importar tudo  
import funcoes  
funcoes.saudacao("Francisco")  
  
# Forma 2: importar específico  
from funcoes import somar  
resultado = somar(5, 3)  
print(resultado)
```

Navegando entre Pastas

Às vezes os módulos ficam em **pastas diferentes**. Entender a estrutura de diretórios é fundamental para projetos maiores.

01

Estrutura de Diretórios

```
meu_projeto/  
|  
├── main.py  
└── utilidades/  
    ├── calculos.py  
    └── mensagens.py
```

02

Código do Módulo

Dentro de `mensagens.py`:

```
def bem_vindo():  
    print("Bem-vindo!")
```

03

Importando de Subpastas

Dentro de `main.py`:

```
from utilidades import mensagens  
  
mensagens.bem_vindo()
```

📌 **Importante:** Esse tipo de estrutura é o início da **modularização profissional** em Python e você irá utilizar ela sempre que trabalhar com POO (orientação a objeto)!

Boas Práticas e Vantagens

✓ Boas Práticas com Módulos

- **Nomes Descritivos**

Dê nomes curtos e claros aos módulos

- **Agrupe Relacionados**

Mantenha módulos similares na mesma pasta

- **Evite Repetição**

Não duplique funções em diferentes módulos

- **Documente Sempre**

Adicione comentários nas funções

◆ Vantagens da Modularização

Organização	Código mais limpo e fácil de entender
Reutilização	Funções podem ser usadas em outros projetos
Manutenção	Erros são mais fáceis de identificar e corrigir
Colaboração	Várias pessoas trabalham em partes diferentes



Atividades Práticas

Exercício 1 - Criando seu Primeiro Módulo

Crie `saudacoes.py` com duas funções:

- `bom_dia()` → imprime "Bom dia!"
- `boa_noite()` → imprime "Boa noite!"

Depois, crie `main.py` que utilize essas funções.

Exercício 2 - Separando Funções em Módulos

Crie `matematica.py` com as funções:

- `somar(a, b)`
- `subtrair(a, b)`
- `multiplicar(a, b)`
- `dividir(a, b)`

Crie `main.py` que use essas funções para calcular com números do usuário.

Exercícios Avançados

Exercício 3 – Estrutura de Loja

```
projeto_loja/  
|  
├── principal.py  
└── utilidades/  
    ├── clientes.py  
    └── produtos.py
```

Crie funções `cadastrar_cliente(nome)` e `listar_produtos()` nos módulos correspondentes.

Exercício 4 – Sistema de Notas

```
notas/  
|  
├── calculos.py  
├── mensagens.py  
└── principal.py
```

Monte um sistema que calcula média de 3 notas e exibe se o aluno foi aprovado ou reprovado.

Desafio Extra – Sistema Bancário

```
sistema_banco/  
|  
├── main.py  
├── contas/  
|   ├── cadastro.py  
|   └── saldo.py  
└── clientes/  
    ├── dados.py  
    └── mensagens.py
```

Monte a estrutura de um sistema bancário. Cada módulo deve ter funções relacionadas ao seu nome:

- `cadastrar_conta()`
- `mostrar_saldo()`
- `mostrar_dados()`
- `mensagem_boas_vindas()`